OPERATIONS ANALYTICS / INDEPENDENT CASE STUDY

International Supply Chain Analytics

Technical project report

Ankith Ramesh Babu

26 October 2026 | Release 2

Order fulfillment, exception prioritization and regional performance.

This report documents a reproducible analysis of synthetic operational records. It connects the business questions, calculation rules, software implementation, dashboard views and verification evidence in one reviewable package.

Scope	Default dataset	Primary outputs
Four warehouse regions	1,200 orders	Python analysis, HTML dashboards, Power BI project source, CSV exports

Results describe the included simulation. They are not employer performance measurements, deployed ERP outcomes or quantified cost savings.

Repository: github.com/ankithr457/international-supply-chain-analytics

Executive summary and contents

The analysis turns order-level stock and fulfillment records into an exception queue and a warehouse scorecard. The central distinction is between unit fill rate and on-time-in-full (OTIF): a partially fulfilled order can contribute shipped units while still failing OTIF.

Result	Value	Interpretation
Blocked orders	106 / 1,200	Administrative release queue
Unit fill rate	78.57%	52,713 / 67,089 unblocked ordered units
OTIF	8.92%	107 complete, unblocked, on-time orders

Report navigation

Section	Page
Business scope and architecture	3
Data specification and provenance	4
Calculation methodology	5
Results and operational interpretation	6
Dashboard overview and interaction	7-8
Power BI model and report layout	9
Sensitivity analysis	10
Implementation excerpts	11
Validation and reproducibility	12
Limitations and conclusion	13
References and artifact index	14

Business scope and architecture

Questions addressed

• Which orders require action, and why?

• How much demand can be fulfilled on unblocked orders?

• Which warehouse has the largest exception workload?

• How does the definition of OTIF change interpretation?

Processing sequence

Stage	Implementation	Output
Generate	Seeded Python simulation	Operational records
Calculate	Documented business rules	KPIs and decision fields
Export	Deterministic CSV / JSON writer	Auditable report files
Present	HTML and PBIR visual definitions	Interactive review surfaces
Verify	Unit tests, schema and browser checks	Validation evidence

Design decisions

The analysis uses Python standard-library functions to keep execution portable. The browser report embeds its data and needs no server. The Power BI model embeds the same CSV snapshot in Power Query, avoiding machine-specific paths. Data generation and decision rules remain separate from presentation.

Operational boundaries

This is a batch analysis of simulated observations. It does not write to SAP, execute procurement transactions, reserve stock or authorize operational changes. Each exported row retains the identifiers needed to trace a result back to its source observation.

Data specification and provenance

All source observations are generated in src/run.py with a fixed random seed. No customer, supplier or employee data is used. Regional names represent four analytical groups, not actual sites.

Field	Type	Meaning
order_id	Text / unique	SO-prefixed order identifier
warehouse	Text	Germany, Hong Kong, Taiwan, Malaysia
ordered_units	Integer	Uniformly sampled from 1 to 120
available_units	Integer	max(0, order quantity + offset -45 to +25)
blocked	Boolean	Independent probability of 9%
delivery_delay_days	Integer	Uniform integer from -2 to 8
fulfilled_units	Integer	0 if blocked, otherwise min(quantity, stock)
status / OTIF	Derived	Priority classification and strict delivery flag

The availability field is a per-order snapshot. The simulation does not share inventory between orders or model a rolling stock ledger. Delivery delay is sampled independently of stock and block status. These simplifications create a controlled exception dataset. Seed 42 produces the committed baseline.

Traceability

Committed CSVs, KPI JSON, analysis JSON and the Power BI embedded snapshot refer to the same default run. Changes to seeds or assumptions require regenerated reports and a new result interpretation.

Calculation methodology

Metric	Definition
Unit fill rate	Fulfilled units / ordered units on unblocked orders
OTIF rate	Unblocked AND full quantity AND delay <= 0, divided by all orders
Shortage units	max(ordered units - available units, 0)
Status priority	Blocked, then Stock shortage, then Late, then On track

Worked example

An unblocked order requests 10 units with 6 available and a delay of -1 day. Fulfilled quantity is 6, shortage is 4 and the order contributes 60% unit fill. OTIF is false because the order is incomplete. If the order is blocked, fulfillment becomes zero and its quantity is excluded from the unit-fill denominator.

Overlapping flags versus exclusive status

The baseline contains 753 orders with an availability shortage, but 687 rows carry the primary status Stock shortage. The remaining 66 shortage orders are blocked and therefore receive the higher-priority Blocked status. Counting primary status and counting shortage flags answer different questions.

Denominator policy

OTIF uses all 1,200 orders and therefore includes the operational effect of blocks. Unit fill rate excludes blocked demand to isolate fulfillable, released demand. These two measures must not be treated as interchangeable service measures.

Results and interpretation

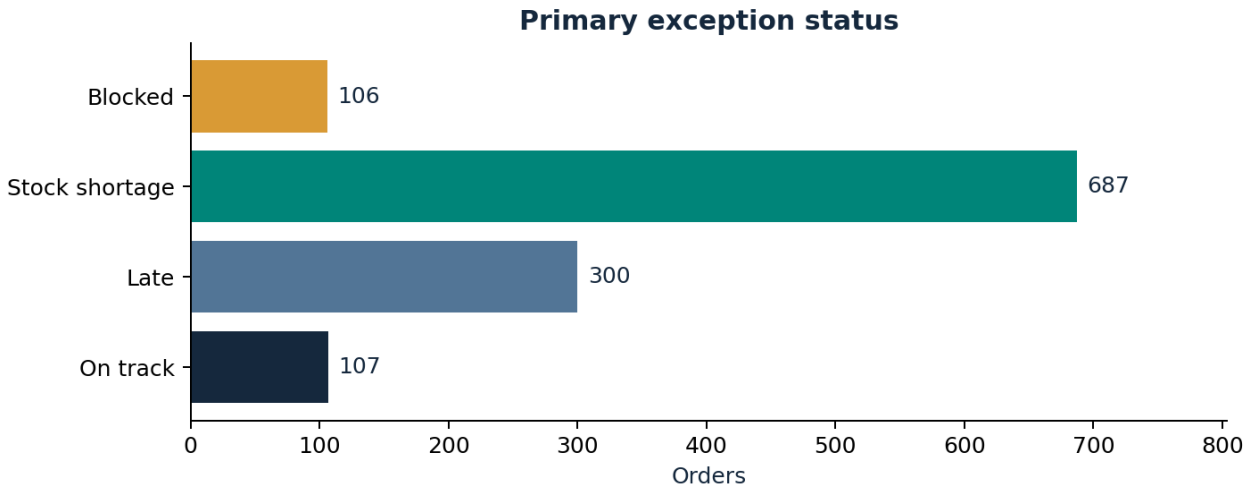


Figure 1. Baseline results calculated from the committed operational records.

Warehouse	Orders	Fill rate	OTIF
Germany	285	79.33%	9.47%
Hong Kong	315	78.94%	8.89%
Taiwan	292	78.18%	7.88%
Malaysia	308	77.86%	9.42%

The main workload is stock shortage: 687 orders carry this primary status. Another 300 complete, unblocked orders are late. Only 107 orders meet the complete, unblocked and on-time condition. The 1,093-row exception register excludes the 107 On track orders.

The low OTIF value follows the deliberately difficult simulation distributions. It is not a benchmark for any company. The practical use is testing whether the exception rules and denominator choices produce an auditable queue.

Interactive dashboard overview

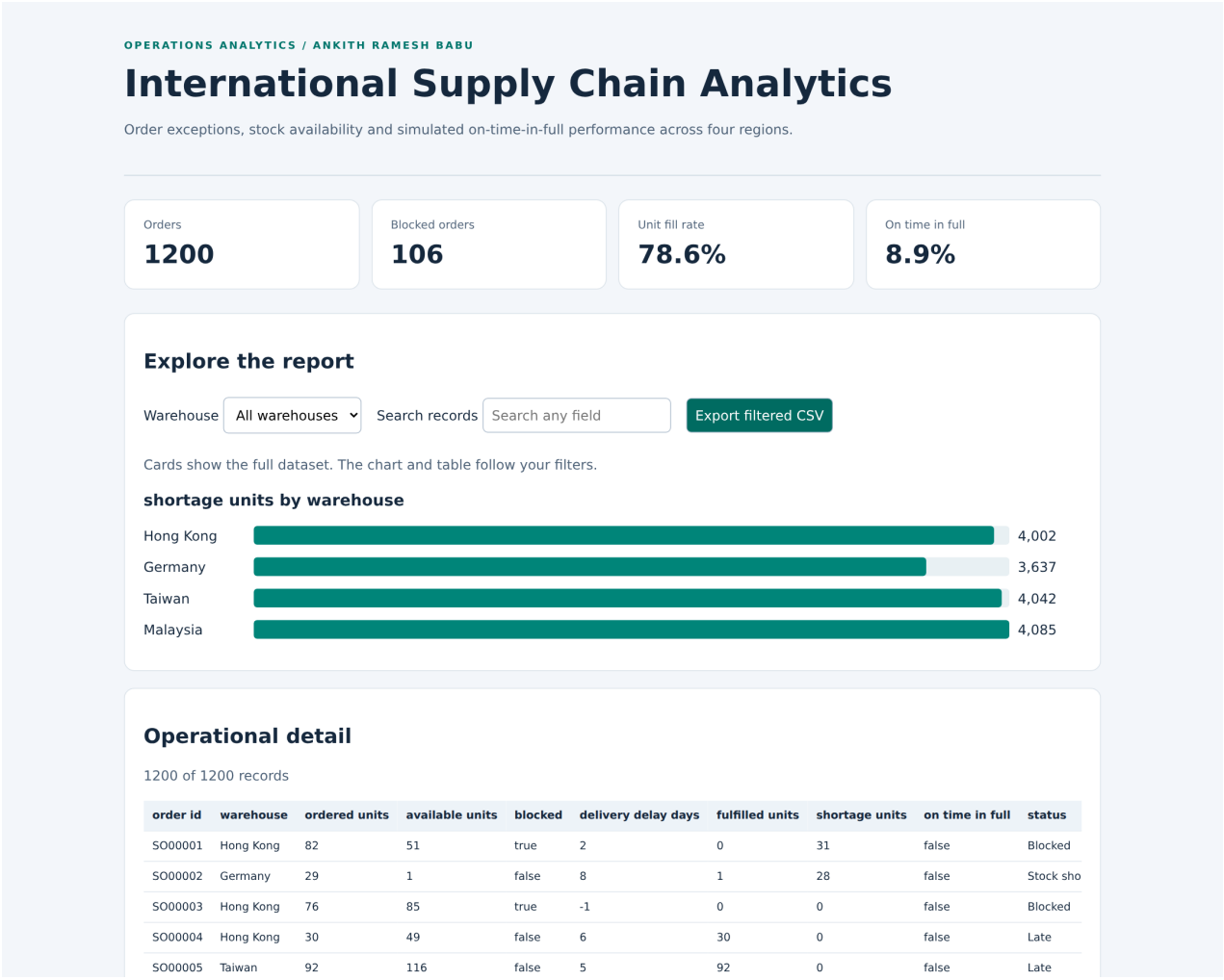


Figure 2. Actual browser capture of the generated HTML dashboard, baseline run.

The overview combines full-dataset KPI cards with warehouse filtering, text search, a comparative bar chart and a detailed register. Its data is embedded in the HTML, so the report opens offline without account access.

Display precision differs from the calculation exports: the HTML KPI cards show one decimal place for ratios, while KPI JSON preserves four decimal places. This is display rounding, not a calculation difference.

Dashboard interaction and audit trail

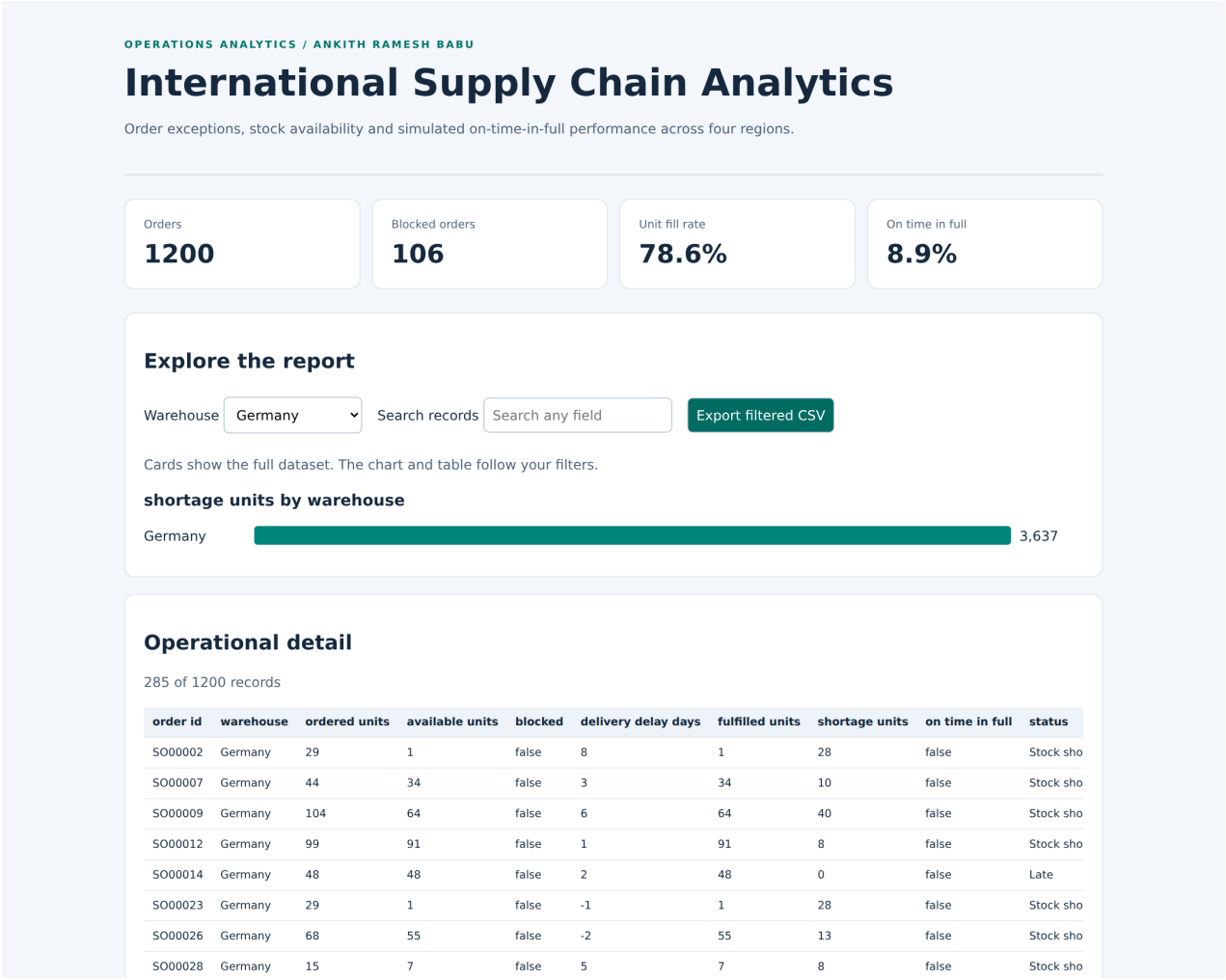


Figure 3. Actual browser capture with the Germany filter applied.

Warehouse and search filters update the chart and visible detail rows. Summary cards intentionally remain scoped to the complete dataset. The table displays at most 200 matching rows for responsiveness. CSV export includes every matching row, not only the displayed sample.

In the browser checks, selecting Germany returned records, an unmatched search returned zero rows, and filtered CSV download succeeded. The layout also passed a 390-pixel mobile-width overflow check. The wide detail table remains horizontally scrollable.

Power BI model and page layout

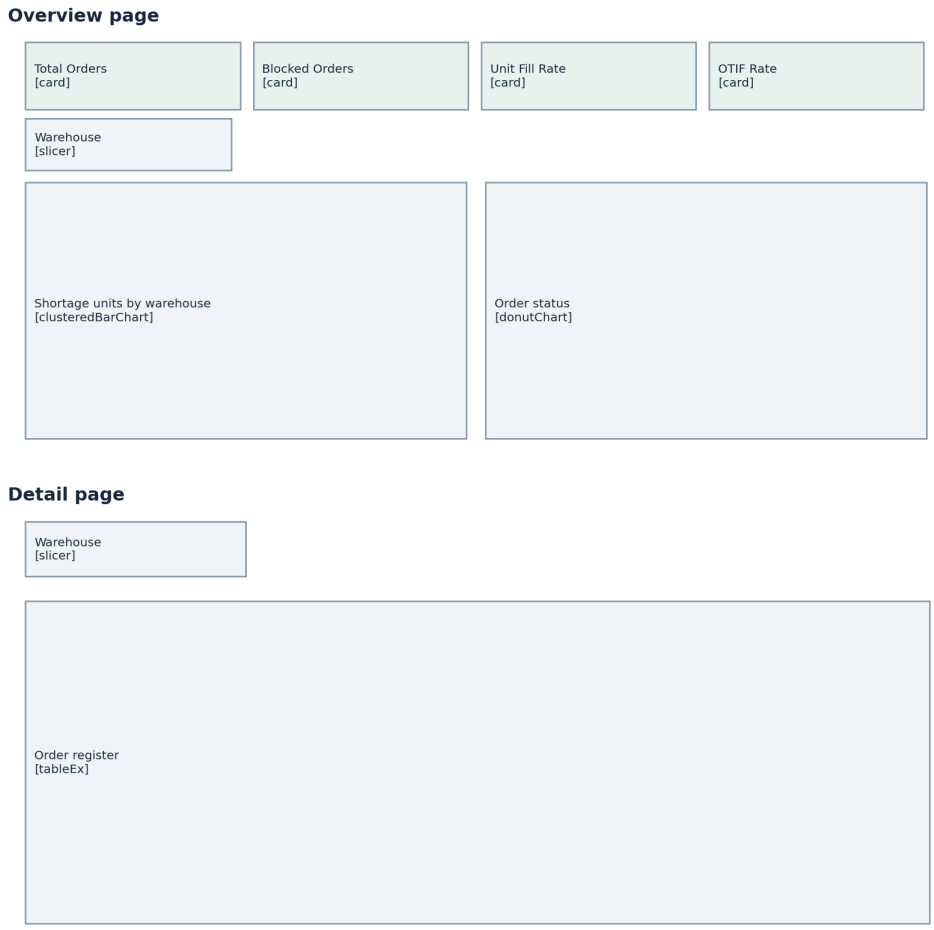


Figure 4. Layout diagram derived from PBIR visual coordinates. This is not a Power BI Desktop screenshot.

The package includes a PBIP entry point, PBIR report pages and a TMSL model.bim semantic model. Power Query decodes an embedded CSV snapshot, promotes headers and applies explicit data types. DAX measures provide aggregation under report filter context.

The overview page groups KPI cards, a warehouse slicer and two charts. The detail page exposes order-level exception fields.

Verification boundary: JSON schema and field-reference checks cover the source package. Power BI Desktop rendering, refresh and DAX execution remain unverified because the Windows application is unavailable in this environment. No PBIX or Desktop screenshot is represented as tested.

Sensitivity and comparative analysis

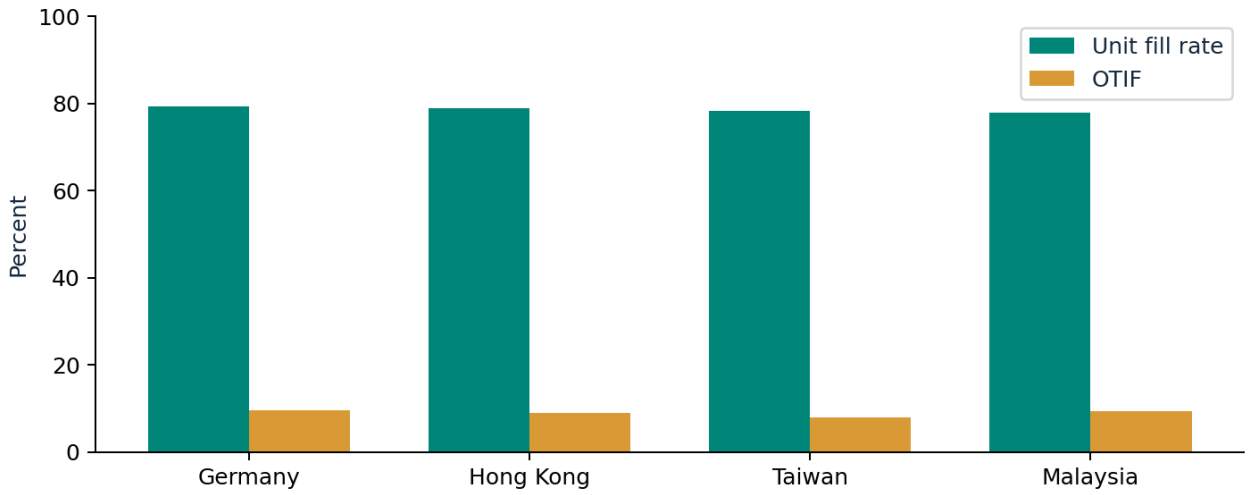


Figure 5. Warehouse-level rates under the baseline simulation.

Seed	Blocked orders	Fill rate	OTIF
7	127	78.26%	8.67%
42	106	78.57%	8.92%
99	124	79.32%	9.75%

The alternate-seed runs retain 1,200 orders and identical distributions. Fill rate ranges from 78.26% to 79.32%, while OTIF ranges from 8.67% to 9.75%. This checks that the analysis works beyond one random draw. It is not a statistical confidence interval or a business forecast.

The warehouse comparison reflects random allocation and sampled conditions. Ranking regions from this artificial dataset is useful for checking scorecard behavior, but not for evaluating real logistics performance.

Implementation excerpts

KPI calculation

```
def metrics(rows):
    total = sum(r["ordered_units"] for r in rows if not r["blocked"])
    filled = sum(r["fulfilled_units"] for r in rows)
    return {"orders": len(rows), "blocked_orders": sum(r["blocked"] for r in
        rows),
            "fill_rate": round(filled / total, 4) if total else 0,
            "otif_rate": round(sum(not r["blocked"] and r["fulfilled_units"] ==
        r["ordered_units"] and r["delivery_delay_days"] <= 0 for r in rows) /
        len(rows), 4) if rows else 0}
```

The denominator explicitly excludes blocked orders for unit fill rate. OTIF requires all three conditions in a single predicate. Empty input produces zero rather than a division error.

Power BI measure

```
OTIF Rate =
DIVIDE(
  CALCULATE(COUNTROWS(orders),
            orders[on_time_in_full] = TRUE()),
  COUNTROWS(orders),
  0
)
```

Code organization

src/run.py owns simulation and business rules. src/reporting.py handles CSV and HTML rendering. scripts/analyze.py creates scenario evidence. scripts/build_powerbi.py regenerates the embedded snapshot and report definitions. Tests exercise calculations, edge cases and report generation.

Validation and reproducibility

Check	Evidence	Outcome
Business-rule tests	Blocked/partial OTIF, quantity bounds and invalid counts	Passed
Repeat generation	Same seed and arguments reproduce identical exports	Passed
Browser interactions	Warehouse filter, empty search, CSV export, mobile width	Passed
Report source	PBIR JSON schema and model-field linkage checks	Passed
Desktop Power BI	Windows application, refresh and DAX engine	Not executed

Reproduction commands

```
python src/run.py
python scripts/analyze.py
python scripts/build_powerbi.py
python -m unittest discover -s tests -v
```

The default analysis uses the Python standard library. Report authoring dependencies are isolated in requirements-report.txt. PDF charts and tables draw from committed CSV and JSON outputs. Browser screenshots record the generated HTML interface rather than a hand-drawn imitation.

Cross-checks

The total primary-status count equals 1,200. Fulfilled units never exceed ordered or available units. Blocked orders always have zero fulfillment. The exception export contains only non-On track rows. The scorecard partitions the same order population.

Scope of assurance

Passing these checks establishes internal consistency for the stated rules and dataset. It does not establish calibration against a real warehouse, production readiness or economic benefit.

Limitations and conclusion

Model limitations

• Stock snapshots are independent across orders, so orders do not compete for a shared balance.

• Delay and block status are independently sampled. Real causes are often correlated.

• No promised calendar date, transport milestone or partial-shipment history exists.

• The scenario analysis changes random seeds rather than estimating real uncertainty.

Next engineering priorities

An operational extension should introduce validated external data, explicit business definitions, schema checks and exception ownership. A continuous stock ledger and dated shipment events would enable a more realistic service model.

Conclusion

The project provides an auditable chain from simulated observations to decision fields, exports and visual review. Its main contribution is a consistent separation of fulfillment, strict OTIF and exception priority.

The reported outputs are reproducible and the calculation boundaries are documented. Power BI source is supplied as an editable project, with Desktop verification explicitly separated from the automated source checks.

References and artifact index

[1] Python Software Foundation. random: Generate pseudo-random numbers.
https://docs.python.org/3/library/random.html

[2] Python Software Foundation. statistics: Mathematical statistics functions.
https://docs.python.org/3/library/statistics.html

[3] Microsoft. Power BI Desktop projects.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview

[4] Microsoft. Power BI project report folder and PBIR.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-report

[5] Microsoft. Power BI project semantic model folder.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset

[6] Microsoft. DIVIDE function (DAX).
https://learn.microsoft.com/en-us/dax/divide-function-dax

Sources consulted 26 October 2026. The inventory policy is an illustrative project assumption. Documentation references support software formats and function behavior, not a claim of real-world model validation.

Artifact index

Path	Purpose
src/	Simulation, calculations and HTML reporting
reports/*.csv, *.json	Baseline and scenario evidence
reports/dashboard.html	Interactive operational dashboard
reports/figures/	Actual browser captures, charts and layout diagram
powerbi/	PBIP, PBIR and semantic model source
tests/	Automated checks
docs/methodology.md	Definitions and assumptions
reports/project-report.pdf	This technical report